

Assignment-12.2

AI ASSISTED CODING

NAME: B. Akhira Nandhini

HTNO: 2303A51516

BATCH: 22

Task Description -1 (Data Structures – Stack Implementation with AI Assistance)

Prompt: Generate a Python program that implements a Stack data structure using a class. Include the methods push(element), pop(), peek(), and is_empty(). Add proper error handling for stack underflow and include clear docstrings for each method explaining their functionality. Take input from user

Code:

```

12:59 > |ipy> % Stack
1 #Generate a Python program that implements a Stack data structure using a
2 #class. Include the methods push(element), pop(), peek(), and is_empty(). Add
3 #proper error handling for stack underflow and include clear docstrings for each
4 #method explaining their functionality. Take input from user.
5 from py_compile import main
6 class Stack:
7     """
8     A class to represent a Stack data structure.
9     """
10    Methods
11    -----
12    push(element):
13        Add an element to the top of the stack.
14    pop():
15        Removes and returns the top element of the stack. Raises an error if the stack is empty.
16    peek():
17        Returns the top element of the stack without removing it. Raises an error if the stack is empty.
18    is_empty():
19        Returns True if the stack is empty, False otherwise.
20    """
21    def __init__(self):
22        """Initialize an empty stack."""
23        self.stack = []
24    def push(self, element):
25        """
26        Add an element to the top of the stack.
27        Parameters
28        -----
29        element : any
30            The element to be added to the stack.
31        """
32        self.stack.append(element)
33    def pop(self):
34        """
35        Remove and return the top element of the stack.
36        Returns
37        -----
38        any
39            The top element of the stack.
40        Raises
41        -----
42        IndexError
43            If the stack is empty (stack underflow).
44        """
45        if self.is_empty():
46            raise IndexError("Stack underflow: Cannot pop from an empty stack.")
47        return self.stack.pop()
48    def peek(self):
49        """
50        Return the top element of the stack without removing it.
51        Returns
52        -----
53        any
54            The top element of the stack.
55        Raises
56        -----
57        IndexError
58            If the stack is empty (stack underflow).
59        """
60        if self.is_empty():
61            raise IndexError("Stack underflow: Cannot peek from an empty stack.")
62        return self.stack[-1]
63    def is_empty(self):
64        """
65        Check if the stack is empty.
66        Returns
67        -----
68        bool
69            True if the stack is empty, False otherwise.
70        """
71        return len(self.stack) == 0
72
73 # Example usage
74 if __name__ == "__main__":
75     stack = Stack()
76     while True:
77         print("\nStack Operations:")
78         print("1. Push")
79         print("2. Pop")
80         print("3. Peek")
81         print("4. Check if Empty")
82         print("5. Exit")
83         choice = input("Enter your choice (1-5): ")
84         if choice == '1':
85             element = input("Enter the element to push: ")
86             stack.push(element)
87             print(f"{element} has been pushed to the stack.")
88         elif choice == '2':
89             try:
90                 popped_element = stack.pop()
91                 print(f"{popped_element} has been popped from the stack.")
92             except IndexError as e:
93                 print(e)
94         elif choice == '3':
95             try:
96                 top_element = stack.peek()
97                 print(f"The top element is: {top_element}")
98             except IndexError as e:
99                 print(e)
100        elif choice == '4':
101            if stack.is_empty():
102                print("The stack is empty.")
103            else:
104                print("The stack is not empty.")
105        elif choice == '5':
106            print("Exiting the program.")
107            break
108        else:
109            print("Invalid choice. Please enter a number between 1 and 5.")
110    if __name__ == "__main__":
111        main()

```

Output:

```
PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding> & C:\Users\abhin\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/abhin/OneDrive/Desktop/AI assisted coding/StackOperations.py"

Stack Operations:
1. Push
2. Pop
3. Peek
4. Check if Empty
5. Exit
Enter your choice (1-5): 1
Enter the element to push: 10
'10' has been pushed to the stack.

Stack Operations:
1. Push
2. Pop
3. Peek
4. Check if Empty
5. Exit
Enter your choice (1-5): 2
'10' has been popped from the stack.

Stack Operations:
1. Push
2. Pop
3. Peek
4. Check if Empty
5. Exit
Enter your choice (1-5): 3
Stack underflow: Cannot peek from an empty stack.

Stack Operations:
1. Push
2. Pop
3. Peek
4. Check if Empty
5. Exit
Enter your choice (1-5): 4
The stack is empty.

Stack Operations:
1. Push
2. Pop
3. Peek
4. Check if Empty
5. Exit
Enter your choice (1-5): 5
Exiting the program.
```

Justification: A Stack is used because it follows the Last In First Out (LIFO) principle, which is suitable for operations where the most recent element must be accessed first. Stacks allow efficient insertion and deletion operations using push and pop in constant time $O(1)$. Other data structures like queues or lists do not strictly follow the LIFO order required for stack-based operations.

Task Description -2 (Algorithms – Linear vs Binary Search Analysis)

Prompt:Generate Python code to implement Linear Search and Binary Search with functions: `linear_search(arr, target)` and `binary_search(arr, target)`. Include Proper docstrings explaining the working principle. Test the algorithms with different input sizes (from user) and print the results for comparison.

Code:

```

#Generate Python code to implement Linear Search and Binary Search with
#functions:
#linear_search(arr, target) and binary_search(arr, target).
#Include Proper docstrings explaining the working principle.Test the algorithms
#with different input sizes (from user) and print the results for comparison.
def linear_search(arr, target):
    """
    Perform a linear search for the target element in the given array.

    Parameters
    -----
    arr : list
        The list in which to search for the target element.
    target : any
        The element to search for in the array.

    Returns
    -----
    int
        The index of the target element if found, otherwise -1.
    """
    for index in range(len(arr)):
        if arr[index] == target:
            return index
    return -1

def binary_search(arr, target):
    """
    Perform a binary search for the target element in the given sorted array.

    Args:
        arr (list): A sorted list in which to search for the target element.
        target (any): The element to search for in the array.

    Returns:
        int: The index of the target element if found, otherwise -1.
    """
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

def main():
    arr = list(map(int, input("Enter a sorted list of integers (space-separated): ").split(' ')))
    target = int(input("Enter the target element to search for: "))

    # Test Linear Search
    linear_result = linear_search(arr, target)
    if linear_result != -1:
        print(f"Linear Search: Target found at index {linear_result}.")
    else:
        print("Linear Search: Target not found in the array.")

    # Test Binary Search
    binary_result = binary_search(arr, target)
    if binary_result != -1:
        print(f"Binary Search: Target found at index {binary_result}.")
    else:
        print("Binary Search: Target not found in the array.")

if __name__ == "__main__":
    main()

```

Output:

```
PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding> & C:\Users\abhin
Enter a sorted list of integers (space-separated): 10,20,30,40,50
Enter the target element to search for: 30
Linear Search: Target found at index 2.
Binary Search: Target found at index 2.
PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding> █
```

Justification: Linear Search and Binary Search use the Array/List data structure because it allows easy sequential and indexed access to elements. Binary Search specifically requires a sorted array to efficiently divide the search space. Other data structures like linked lists or trees are not suitable because they do not support efficient direct indexing required for Binary Search.

Task Description -3 (Test Driven Development – Simple Calculator Function)

Prompt:

Generate unit test cases first for a simple calculator in Python using unittest. The calculator should support two operations: addition and subtraction. Take input from user for the numbers and the operation to perform. Create a separate test file named test_calculator.py. Do not implement the calculator functions yet.

Code:

```
import unittest
from calculator import add, subtract
class TestCalculator(unittest.TestCase):
    """Unit test cases for the simple calculator functions."""

    def test_add(self):
        """Test the addition function."""
        self.assertEqual(add(2, 3), 5)
        self.assertEqual(add(-1, 1), 0)
        self.assertEqual(add(0, 0), 0)

    def test_subtract(self):
        """Test the subtraction function."""
        self.assertEqual(subtract(5, 2), 3)
        self.assertEqual(subtract(0, 5), -5)
        self.assertEqual(subtract(-1, -1), 0)
if __name__ == '__main__':
    unittest.main()
```

Now implement the calculator functions `add()` and `subtract()` in `calculator.py` so that all the existing unit tests pass.

```

import unittest
from calculator import add, subtract

class TestCalculator(unittest.TestCase):

    def test_add_positive(self):
        self.assertEqual(add(5, 3), 8)

    def test_add_negative(self):
        self.assertEqual(add(-2, -3), -5)

    def test_add_zero(self):
        self.assertEqual(add(0, 5), 5)

    def test_subtract_positive(self):
        self.assertEqual(subtract(10, 4), 6)

    def test_subtract_negative(self):
        self.assertEqual(subtract(-5, -2), -3)

    def test_subtract_zero(self):
        self.assertEqual(subtract(5, 0), 5)

if __name__ == "__main__":
    unittest.main()

```

Output:

```

..
-----
Ran 2 tests in 0.001s

OK
PS C:\Users\AdepuTejaswini\AI assist> python -m unittest test_calculator.py
.....
-----
Ran 6 tests in 0.001s

OK
PS C:\Users\AdepuTejaswini\AI assist> 

```

Justification:

Test Driven Development ensures that test cases are written before implementing the actual functionality. It helps verify that the program meets requirements and reduces bugs during development. Using AI tools like Copilot speeds up writing tests and implementation while maintaining code correctness.

Task Description -4 (Data Structures – Queue Implementation with AI Assistance)

Prompt: Generate a Python program that implements a Queue data structure using a class. Include methods enqueue(element), dequeue(), front(), and is_empty(). Handle queue overflow and underflow conditions and include proper docstrings for each method. Take input from user to perform queue operations.

Code:


```

class Queue:
    """A class representing a Queue data structure."""

    def __init__(self):
        """Initialize an empty queue."""
        self.queue = []

    def enqueue(self, element):
        """Add an element to the rear of the queue.

        Args:
            element: The element to be added to the queue.
        """
        self.queue.append(element)

    def dequeue(self):
        """Remove and return the front element of the queue.

        Returns:
            The front element of the queue.

        Raises:
            IndexError: If the queue is empty (queue underflow).
        """
        if self.is_empty():
            raise IndexError("Queue underflow: Cannot dequeue from an empty queue.")
        return self.queue.pop(0)

    def front(self):
        """Return the front element of the queue without removing it.

        Returns:
            The front element of the queue.

        Raises:
            IndexError: If the queue is empty (queue underflow).
        """
        if self.is_empty():
            raise IndexError("Queue underflow: Cannot access front of an empty queue.")
        return self.queue[0]

    def is_empty(self):
        """Check if the queue is empty.

        Returns:
            True if the queue is empty, False otherwise.
        """
        return len(self.queue) == 0

def main():
    queue = Queue()
    while True:
        print("\nQueue Operations:")
        print("1. Enqueue")
        print("2. Dequeue")
        print("3. Front")
        print("4. Check if Empty")
        print("5. Exit")

        choice = input("Enter your choice (1-5): ")

        if choice == '1':
            element = input("Enter the element to enqueue: ")
            queue.enqueue(element)
            print(f'{element}' has been enqueued to the queue.")
        elif choice == '2':
            try:
                dequeued_element = queue.dequeue()
                print(f'{dequeued_element}' has been dequeued from the queue.")
            except IndexError as e:
                print(e)
        elif choice == '3':
            try:
                front_element = queue.front()
                print(f"The front element is: '{front_element}'")
            except IndexError as e:
                print(e)
        elif choice == '4':
            if queue.is_empty():
                print("The queue is empty.")
            else:
                print("The queue is not empty.")
        elif choice == '5':
            print("Exiting the program.")
            break
        else:
            print("Invalid choice. Please enter a number between 1 and 5.")

if __name__ == "__main__":
    main()

```

Output:

```
Queue Operations:
1. Enqueue
2. Dequeue
3. Front
4. Check if Empty
5. Exit
Enter your choice (1-5): 1
Enter the element to enqueue: 10
'10' has been enqueued to the queue.

Queue Operations:
1. Enqueue
2. Dequeue
3. Front
4. Check if Empty
5. Exit
Enter your choice (1-5): 1
Enter the element to enqueue: 20
'20' has been enqueued to the queue.

Queue Operations:
1. Enqueue
2. Dequeue
3. Front
4. Check if Empty
5. Exit
Enter your choice (1-5): 1
Enter the element to enqueue: 30
'30' has been enqueued to the queue.

Queue Operations:
1. Enqueue
2. Dequeue
3. Front
4. Check if Empty
5. Exit
Enter your choice (1-5): 4
The queue is not empty.

Queue Operations:
1. Enqueue
2. Dequeue
3. Front
4. Check if Empty
5. Exit
Enter your choice (1-5): 2
'10' has been dequeued from the queue.

Queue Operations:
1. Enqueue
2. Dequeue
3. Front
4. Check if Empty
5. Exit
Enter your choice (1-5): 3
The front element is: '20'

Queue Operations:
1. Enqueue
2. Dequeue
3. Front
4. Check if Empty
5. Exit
Enter your choice (1-5): 5
Exiting the program.
```

Justification: Queue is suitable because it follows the FIFO (First In First Out) principle, which is ideal for processing elements in the order they arrive. It is commonly used in task scheduling, buffering, and request handling systems. Other data structures like Stack (LIFO) or Linked List do not naturally maintain FIFO order for processing elements.

Task Description -5 (Algorithms – Bubble Sort vs Selection Sort)

Prompt: Write Python functions `bubble_sort(arr)` and `selection_sort(arr)` with comments explaining each step. Include docstrings mentioning time complexity and space complexity. Return the sorted array and demonstrate the functions with sample test cases. Take input from user for the array to be sorted.

Code:

```
def bubble_sort(arr):
    """Sort an array using the Bubble Sort algorithm.

    Args:
        arr: A list of elements to be sorted.

    Returns:
        A sorted list of elements.

    Time Complexity: O(n^2) in the worst and average cases, O(n) in the best case (when the array is already sorted).
    Space Complexity: O(1) (in-place sorting).
    """
    n = len(arr)
    # Traverse through all elements in the array
    for i in range(n):
        # Last i elements are already in place, no need to check them
        for j in range(0, n - i - 1):
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

def selection_sort(arr):
    """Sort an array using the Selection Sort algorithm.

    Args:
        arr: A list of elements to be sorted.

    Returns:
        A sorted list of elements.

    Time Complexity: O(n^2) in all cases (best, average, and worst).
    Space Complexity: O(1) (in-place sorting).
    """
    n = len(arr)
    # Traverse through all elements in the array
    for i in range(n):
        # Find the minimum element in the remaining unsorted array
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        # Swap the found minimum element with the first element of the unsorted array
        arr[i], arr[min_index] = arr[min_index], arr[i]
    return arr

def main():
    arr = input("Enter a list of elements to sort (comma separated): ").split(',')
    print("Original array:", arr)

    sorted_arr_bubble = bubble_sort(arr.copy())
    print("Sorted array using Bubble Sort:", sorted_arr_bubble)

    sorted_arr_selection = selection_sort(arr.copy())
    print("Sorted array using Selection Sort:", sorted_arr_selection)

if __name__ == "__main__":
    main()
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding> & C:\Users\abhin\AppData\Local\
Enter a list of elements to sort (comma separated): 5,8,6,4,7,1,9
Original array: ['5', '8', '6', '4', '7', '1', '9']
Sorted array using Bubble Sort: ['1', '4', '5', '6', '7', '8', '9']
Sorted array using Selection Sort: ['1', '4', '5', '6', '7', '8', '9']
PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding> █
```

Justification: Sorting algorithms like Bubble Sort and Selection Sort require indexed access, which is efficiently supported by arrays (Python lists). Arrays allow easy swapping of elements using indices, which is essential for these algorithms. Other data structures like linked lists, stacks, or queues are inefficient for direct element comparison and swapping, making arrays the most suitable choice.